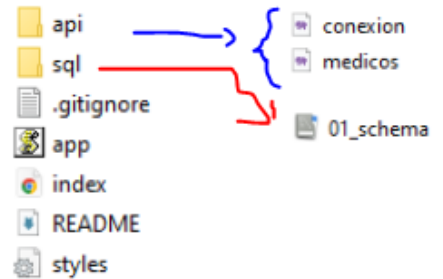


Recorrido completo: Documentos médicos

1 Reconocimiento del proyecto

1. ¿Qué archivos y carpetas forman el proyecto?
2. ¿Qué responsabilidad tiene cada archivo?



index.html: donde se muestra la estructura principal de la página web

styles.css: el archivo que modifica el estilo visual de la página web

app.js: lógica del frontend. Realiza las solicitudes a la API, procesa las respuestas y muestra los documentos en la tabla

medicos.php: funciona como endpoint de la API. Recibe las solicitudes y realiza las consultas necesarias a la base de datos

conexion.php: establece la conexión con MySQL utilizando PDO

01_schema.sql: contiene las instrucciones SQL para crear la base de datos, la tabla y los datos iniciales

README.md: contiene información sobre el proyecto y las instrucciones para que los usuarios entiendan cómo funciona la página web

.gitignore: indica qué archivos o carpetas deben ser ignorados por Git

3. ¿Cuál es el archivo que se muestra primero en el navegador?

El primer archivo que se muestra es el [index.html](#) funcionando como puente entre el cliente y la base de datos

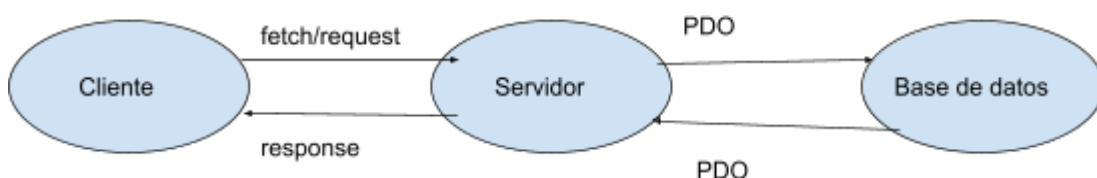
4. ¿Qué archivo contiene la lógica de presentación visual?

El archivo que contiene la lógica visual es el archivo [app.js](#) (java script)

5. ¿Qué archivo contiene la lógica de conexión con la base de datos?

EL archivo que contiene la conexión con la base de datos es [conexión.php](#)

6. Dibujar el recorrido de una solicitud desde el navegador hasta MySQL y de regreso.



2 Recorrido por el frontend

Abrir Index.html y identificar zonas principales

Encabezado: contiene el título "Documentos médicos" y la información principal de la aplicación.

Zona de acciones: contiene el botón "Actualizar listado", que permite volver a solicitar los documentos a la API.

Contador: muestra la cantidad de documentos registrados.

Tabla de documentos: es la zona principal donde se muestran los documentos obtenidos desde la API.

Estado de carga y mensajes: se utilizan para informar al usuario cuando los documentos se están cargando o cuando ocurre algún error.

Zona inferior: contiene información adicional de la aplicación.

¿Qué elementos HTML se utilizan para mostrar el listado?

Para mostrar el listado de documentos se utiliza una tabla `<table>`

¿Qué elemento funciona como cuerpo de la tabla?

El cuerpo de la tabla es `<tbody id="documentsTableBody"></tbody>`

Buscar en app.js la función que carga los documentos. ¿En qué momento se ejecuta?

La función que carga los documentos es `loadDocuments()`.

Se ejecuta automáticamente cuando se carga la página y también cuando el usuario presiona el botón "Actualizar listado"

¿Qué hace `fetch()` y qué URL consulta?

`fetch()` permite realizar una solicitud HTTP desde JavaScript hacia la API para obtener los documentos. URL: `http://localhost/api/medicos.php?action=getall`

¿Qué diferencia hay entre `response` y `result`?

`response` representa la respuesta HTTP recibida después de realizar la solicitud con `fetch()`. Permite comprobar, por ejemplo, si la solicitud fue exitosa. `result` contiene los datos de la respuesta después de convertir el JSON a un objeto de JavaScript.

¿Qué ocurre si la respuesta HTTP no es correcta?

Si la respuesta HTTP no es correcta, o si la API indica que la operación no fue exitosa, JavaScript detecta el error y ejecuta el bloque correspondiente para manejarlo. El usuario recibe un mensaje indicando que ocurrió un problema al cargar los documento

¿Qué mensaje ve el usuario mientras se espera la respuesta?

Mientras se espera la respuesta de la API, el usuario ve el mensaje: "Cargando documentos..."

¿Cómo se actualiza el contador de documentos?

El contador se actualiza utilizando la cantidad de documentos recibidos
`documentCount.textContent = documents.length;`

¿Por qué se utiliza `escapeHtml()` antes de insertar datos en la tabla?

Se utiliza `escapeHtml()` para evitar que los datos recibidos sean interpretados como código HTML o JavaScript. Ayuda a proteger la aplicación frente a posibles ataques y permite que los datos se muestren de forma segura como texto

¿Qué sucede cuando se pulsa "Ver PDF ficticio"?

Cuando se pulsa "Ver PDF ficticio", no se abre un PDF real. JavaScript evita que el enlace sea seguido y nos muestra un mensaje informando que el archivo todavía no está alojado

Modificar temporalmente el texto del botón de actualización y comprobar el resultado.

Si cambiamos el texto del botón de actualización la modificación se ve en la página web pero no afecta en nada

3 Investigación de la API

¿Qué estructura tiene la respuesta JSON?

Cuando todo funciona correctamente, la API devuelve:

```
{
  "success": true,
  "data": [
    {
      "id": 3,
      "nombre": "ecografia_abdominal_2026.pdf",
      "especialista": "Dra. Julia Méndez - Gastroenterología",
      "indicacion": "Investigar molestias abdominales recurrentes.",
      "url_ficticia": "/archivos-ficticios/ecografia_abdominal_2026.pdf"
    }
  ]
}
```

¿Qué significan las propiedades `success` y `data`?

success indica si la operación fue exitosa.

data contiene los documentos obtenidos desde MySQL

¿Qué campos contiene cada documento?

Cada documento contiene:

- id
- nombre
- especialista
- indicacion
- url_ficticia

Además, la tabla SQL tiene activo y creado_en, aunque estos dos campos no son enviados actualmente por la API.

¿Qué método HTTP acepta el endpoint?

El endpoint acepta: GET

También responde a OPTIONS para gestionar la solicitud previa de CORS.

¿Qué ocurre si se consulta sin action=getall?

La API devuelve:

HTTP 400

¿Qué ocurre si se envía una solicitud POST?

El endpoint solo permite GET. Por eso una solicitud POST devuelve: HTTP 405-Method Not Allowed.

Indicando también que se debe usar GET

¿Qué significan los códigos HTTP 200, 204, 400, 405 y 500?

Código	Significado	En esta aplicación
200	OK	La consulta se realizó correctamente.
204	No Content	Se utiliza para responder a OPTIONS.
400	Bad Request	La acción enviada no es válida.
405	Method Not Allowed	Se utilizó un método no permitido, como POST.
500	Internal Server Error	Ocurrió un error interno, por ejemplo, al conectar con MySQL.

¿Para qué sirven las cabeceras Content-Type y Access-Control-Allow-Origin?

Content-Type Indica qué tipo de contenido devuelve la API

Access-Control-Allow-Origin: permite solicitudes desde diferentes orígenes

¿Por qué existe una respuesta para el método OPTIONS?

OPTIONS se utiliza para comprobar qué métodos y condiciones permite el servidor.

```
if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') {  
    http_response_code(204);  
    exit;  
}
```

Respondiendo con 204.

Captura F12 Network

Name	×	Headers	Payload	Preview	Response	Initiator	Timing	Adblock
index.html	✖	General						
styles.css	✓							
index.html	📄	Request URL			http://localhost/api/medicos.php?action=getall			
styles.css	✓	Request Method			GET			
app.js	📄	Status Code			200 OK			
medicos.php?action=getall	🔗	Remote Address			[::1]:80			
index.html	📄	Referrer Policy			strict-origin-when-cross-origin			
styles.css	✓	Response headers						
app.js	📄	Access-Control-Allow-Headers			Content-Type			
medicos.php?action=getall	🔗	Access-Control-Allow-Methods			GET, OPTIONS			
index.html	📄	Access-Control-Allow-Origin			*			
styles.css	✓	Connection			Keep-Alive			
app.js	📄	Content-Length			737			
medicos.php?action=getall	🔗	Content-Type			application/json; charset=utf-8			
		Date			Wed, 26 Aug 2026 22:10:44 GMT			
		Keep-Alive			timeout=5, max=99			
		Server			Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.0.30			
		X-Powered-By			PHP/8.0.30			

4 Conexión con PDO

Leer `api/conexion.php` e identificar dónde se define el host, puerto, base, usuario y contraseña.

```
<?php
declare(strict_types=1);

function obtenerConexion(): PDO
{
    $host = getenv('DB_HOST') ? '127.0.0.1';
    $port = getenv('DB_PORT') ? '3306';
    $database = getenv('DB_NAME') ? 'medicos_demo';
    $username = getenv('DB_USER') ? 'root';
    $password = getenv('DB_PASSWORD') ? 'root';

    $dsn = "mysql:host={$host};port={$port};dbname={$database};charset=utf8mb4";

    return new PDO($dsn, $username, $password, [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES => false,
    ]);
}
```

DB_HOST → host

DB_PORT → puerto

DB_NAME → nombre de la base

DB_USER → usuario

DB_PASSWORD → contraseña

¿Qué es un DSN?

Un DSN (Data Source Name) es una cadena que contiene los datos necesarios para indicarle a PDO cómo y dónde conectarse a la base de datos. En este proyecto se especifica que se utilizará MySQL, el host, el puerto, la base de datos y la codificación utf8mb4.

¿Qué ventaja ofrece PDO frente a conectarse usando funciones específicas de MySQL?

La principal ventaja de PDO es que proporciona una interfaz común para trabajar con diferentes sistemas de bases de datos y ofrece herramientas como consultas preparadas y manejo de errores mediante excepciones. Esto hace que el código sea más flexible, seguro y fácil de mantener.

¿Qué efecto tiene PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION?

PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION hace que PDO genere una excepción cuando ocurre un error en la conexión o en una operación con la base de datos. Esto permite manejar el error mediante try and catch y devolver una respuesta adecuada sin mostrar detalles internos al usuario.

¿Qué significa PDO::FETCH_ASSOC?

PDO::FETCH_ASSOC hace que los resultados de las consultas se devuelvan como arrays asociativos, utilizando el nombre de cada columna como clave. Esto permite acceder a los datos de forma clara, por ejemplo \$fila['nombre'].

¿Por qué se utiliza charset=utf8mb4?

Se utiliza charset=utf8mb4 para establecer la codificación de caracteres de la conexión. Permite trabajar correctamente con caracteres especiales, símbolos, etc. Evitando problemas de codificación entre PHP y MySQL.

Investigar qué son las variables de entorno DB_HOST, DB_PORT, DB_NAME, DB_USER y DB_PASSWORD.

Las variables de entorno DB_HOST, DB_PORT, DB_NAME, DB_USER y DB_PASSWORD almacenan los datos necesarios para conectarse a MySQL sin tener que escribirlos directamente en el código. En este proyecto se obtienen mediante getenv() (*función de PHP que permite obtener el valor de una variable de entorno*) y, si no están definidas, se utilizan valores predeterminados.

Cambiar temporalmente el nombre de la base de datos y observar el comportamiento de la API. Luego restaurarlo.

```
{"success":false,"message":"No fue posible consultar los documentos."}
```

Al cambiar temporalmente el nombre de la base de datos por uno inexistente, la conexión con MySQL falla. PDO genera una excepción debido a PDO::ERRMODE_EXCEPTION, que es capturada por el catch de medicos.php. La API responde con el código HTTP 500 (*error interno del servidor*) y un mensaje genérico indicando que no fue posible consultar los documentos.

¿Qué información sería peligroso mostrar directamente al usuario si la conexión fallara?

No se deberían mostrar credenciales, datos de conexión, rutas internas ni mensajes detallados de PDO o MySQL, porque podrían revelar información sensible sobre el servidor y facilitar ataques. En este proyecto se devuelve un mensaje genérico al usuario en lugar de mostrar los detalles internos del error.

Proponer una forma más segura de administrar las credenciales en un entorno real.

En un entorno real, las credenciales deberían almacenarse fuera del código fuente, por ejemplo mediante variables de entorno o un gestor seguro de secretos. De esta forma, la contraseña y otros datos sensibles no quedan escritos directamente en los archivos de la aplicación. En este proyecto ya se utiliza `getenv()` para obtener estas credenciales desde variables de entorno.

5 Recorrido por SQL

Leer `sql/01_schema.sql` y localizar la creación de la base de datos.

```
CREATE DATABASE IF NOT EXISTS medicos_demo
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;
```

La base de datos se crea al comienzo del archivo mediante `CREATE DATABASE IF NOT EXISTS medicos_demo`.

¿Qué función cumple `IF NOT EXISTS`?

`IF NOT EXISTS` comprueba si la base de datos ya existe antes de crearla. Si no existe, se crea, si ya existe, MySQL no la vuelve a crear y evita un error de duplicación.

¿Qué columnas tiene `documentos_medicos`?

La tabla `documentos_medicos` tiene siete columnas: `id`, `nombre`, `especialista`, `indicacion`, `url_ficticia`, `activo` y `creado_en`.

¿Cuál es la clave primaria y para qué sirve?

La clave primaria de `documentos_medicos` es `id`. Su función es identificar de forma única cada documento dentro de la tabla y evitar que existan dos registros con el mismo identificador.

¿Qué tipo de dato se utiliza para la indicación médica? ¿Por qué?

La columna `indicacion` utiliza el tipo de dato `TEXT`, ya que permite almacenar texto, en este caso indicaciones médicas, de longitud variable sin establecer una cantidad fija de caracteres.

¿Qué representa `activo`?

`Activo` representa el estado del documento dentro del sistema. El valor 1 indica que está activo y puede aparecer en el listado, mientras que 0 indica que está inactivo. La API solo muestra los documentos que tienen `activo = 1`.

¿Qué valor se genera automáticamente en creado_en?

creado_en registra automáticamente la fecha y hora en que se crea cada documento. De esta manera, no es necesario introducir manualmente ese dato al insertar un nuevo registro.

Ejecutar una consulta para contar los documentos activos.

Su consulta se ejecutó con éxito.

```
SELECT COUNT(*) AS cantidad FROM documentos_medicos WHERE activo = 1;
```

☐ Perfilando [\[Editar en línea \]](#) [\[Editar \]](#) [\[Explicar SQL \]](#) [\[Crear código PHP \]](#) [\[Actualizar \]](#)

Opciones extra

cantidad

6

Para contar los documentos activos se utiliza una consulta `SELECT COUNT(*)` filtrando por `activo = 1`. El resultado indica la cantidad total de documentos que actualmente están activos.

Agregar un cuarto documento con datos ficticios y comprobar que aparece en el frontend.

✓ 1 fila insertada
La id de la fila insertada es: 11 (La consulta tardó 0,0001 segundos.)

```
INSERT INTO documentos_medicos (nombre, especialista, indicacion, url_ficticia) VALUES ('radiografia_de_tórax_2026.pdf','Dr. Thiago Blanco','Realizar estudio radiológico según indicación médica.', '/archivos-ficticios/radiografia_de_tórax_2026.pdf');
```

REGISTRO			
Listado de archivos			Listado actualizado
ARCHIVO	ESPECIALISTA	INDICACIÓN	
radiografia_de_tórax_2026.pdf Documento PDF	Dr. Thiago Blanco	Realizar estudio radiológico según indicación médica.	Ver PDF ficticio

Marcar un documento como inactivo. ¿Sigue apareciendo? Explicar por qué.

✓ 1 fila afectada. (La consulta tardó 0,0001 segundos.)

```
UPDATE documentos_medicos SET activo = 0 WHERE id = 4;
```

[[Editar en línea](#)] [[Editar](#)] [[Crear código PHP](#)]

No. Al cambiar activo = 0, el documento deja de aparecer en el frontend porque la API solo consulta los registros cuyo campo activo tiene el valor 1. El registro no se elimina de la base de datos, simplemente queda marcado como inactivo.

Cambiar el orden de la consulta y observar cómo cambia el listado.

```
try {
    require_once __DIR__ . '/conexion.php';
    $conexion = obtenerConexion();
    $consulta = $conexion->query(
        'SELECT id, nombre, especialista, indicacion, url_ficticia
        FROM documentos_medicos
        WHERE activo = 1
        ORDER BY creado_en ASC, id ASC'
    );
}
```

La consulta utiliza ORDER BY creado_en DESC, id DESC, por lo que los documentos aparecen primero desde el más reciente al más antiguo. Al cambiar DESC por ASC, el orden se invierte y los documentos aparecen desde el más antiguo al más reciente.

6 Recorrido por el frontend

¿Qué diferencia existe entre abrir el HTML desde Apache y abrirlo con doble clic?

Cuando el HTML se abre desde Apache, la página se carga mediante el protocolo HTTP, por ejemplo, `http://localhost/`, permitiendo que el navegador se comunique correctamente con la API. En cambio, al abrir el archivo directamente con doble clic, se utiliza el protocolo `file://`. En este caso, la aplicación detecta que está siendo ejecutada de esta manera mediante `window.location.protocol === 'file:'` y utiliza la dirección `http://localhost/api/medicos.php?action=getall` para realizar la consulta a la API.

Detener Apache y observar el mensaje de error del frontend.

Si Apache está detenido, fetch() no podrá comunicarse con la API ubicada en <http://localhost/api/medicos.php?action=getall>, por lo que la solicitud fallará. Como consecuencia, el frontend mostrará el mensaje "Error de conexión" y la tabla mostrará el mensaje correspondiente indicando que no se pudieron cargar los documentos

Apagar MySQL y consultar la API. ¿Qué respuesta se obtiene?

Si MySQL está apagado, PHP podrá recibir la solicitud realizada a la API, pero PDO no podrá establecer la conexión con la base de datos. El error será capturado por el bloque catch y la API responderá con un código HTTP 500

Revisar la pestaña Network del navegador durante una carga exitosa.

Durante una carga exitosa, en la pestaña Network del navegador se puede observar una solicitud con el método GET, un código de respuesta 200 OK y la URL [/api/medicos.php?action=getall](http://localhost/api/medicos.php?action=getall). Al revisar la respuesta de esta solicitud, se puede comprobar que la API devuelve los documentos en formato JSON, demostrando que la comunicación entre el frontend y el backend funciona correctamente

¿Qué datos permiten distinguir un error de frontend de un error de backend?

Un error de frontend generalmente está relacionado con problemas en JavaScript, como errores al ejecutar funciones, modificar elementos HTML o procesar la respuesta, y puede observarse principalmente en la consola del navegador. Un error de backend puede identificarse mediante códigos HTTP como 400, 405 o 500, errores producidos por PHP, problemas de conexión con MySQL o respuestas JSON que contienen success: false

Probar la API con curl y comparar la respuesta con la del navegador.

curl -i "<http://localhost/api/medicos.php?action=getall>" Se obtiene una respuesta HTTP 200 junto con el JSON de los documentos.

Consultar la API con una acción inválida

La API devuelve: HTTP 400 — Bad Request porque la acción indicada no existe.

Tabla de pruebas

Prueba	URL / acción	Resultado esperado	Resultado obtenido
Carga normal	action=getall	HTTP 200 + documentos	Lista de documentos
Acción inválida	action=prueba	HTTP 400	Acción no válida
POST	POST a medicos.php	HTTP 405	Método no permitido
OPTIONS	OPTIONS a medicos.php	HTTP 204	Sin contenido
MySQL apagado	action=getall	HTTP 500	No fue posible consultar
Apache apagado	localhost	Error de conexión	No se puede acceder a la API

7 Seguridad y diseño

¿Qué riesgos tendría permitir cualquier origen con Access-Control-Allow-Origin: * en una aplicación real?

Access-Control-Allow-Origin: * permite que cualquier origen pueda realizar solicitudes a la API desde el navegador. En una aplicación real, esto puede ser riesgoso porque no restringe el acceso a sitios autorizados. Por seguridad, sería preferible permitir únicamente los dominios que realmente necesite utilizar la API.

¿Por qué los datos se escapan antes de insertarse en el HTML?

Los datos se escapan antes de insertarlos en el HTML para evitar que caracteres especiales sean interpretados como código HTML o JavaScript. Esto ayuda a prevenir ataques de inyección como XSS y permite mostrar los datos como texto de forma segura.

¿Qué pasaría si el nombre del archivo contuviera etiquetas HTML?

Si el nombre del archivo contuviera etiquetas HTML, `escapeHtml()` convertiría sus caracteres especiales para que fueran interpretados como texto y no como código HTML. De esta forma se evita que un contenido malicioso pueda ejecutarse en el navegador.

¿Qué información adicional debería validarse antes de guardar un documento?

Antes de guardar un documento se deberían validar todos los campos para comprobar que no estén vacíos cuando corresponda, que tengan el formato y longitud esperados y que sus valores sean válidos. También se deberían rechazar datos inesperados o maliciosos para evitar errores y problemas de seguridad.

¿Por qué los PDFs ficticios no deben confundirse con archivos realmente disponibles?

Los PDFs ficticios no deben confundirse con archivos reales porque en este proyecto solamente representan una referencia o demostración de cómo funciona el enlace. Actualmente los archivos no están alojados y al hacer clic se muestra un mensaje informando al usuario.

Proponer una alternativa para mostrar un enlace deshabilitado en vez de usar un href ficticio.

Una alternativa sería reemplazar el enlace `<a>` con un elemento como `<span>` que indique que el PDF no está disponible. De esta forma se evita utilizar un href ficticio y se comunica claramente al usuario que el archivo todavía no puede abrirse.

¿Qué cambios de accesibilidad se podrían agregar a la tabla y a los mensajes de estado?

Como posibles mejoras, se podría verificar el contraste de colores, asegurar un indicador de foco visible para la navegación mediante teclado y mejorar los estados visuales de los elementos interactivos.

8Desafío integrador

Mejora: agregar la fecha de creación al listado

1. Descripción del problema que resuelve

Se agrega la fecha de creación de cada documento para que el usuario pueda saber cuándo fue registrado.

2. Archivos modificados

[api/medicos.php](#), [index.html](#) y [app.js](#).

3. Solicitud HTTP involucrada

Se mantiene la solicitud GET a `api/medicos.php?action=getall`.

4. Consulta SQL involucrada

Se incluye el campo `creado_en` en la consulta que obtiene los documentos.

5. Captura o evidencia

INDICACION	FECHA	
Investigar molestias abdominales recurrentes.	2026-08-26 20:54:25	Ver PDF ficticio
Control anual de parámetros hematológicos y metabólicos.	2026-08-26 20:54:25	Ver PDF ficticio

```
<td><span class="indication">${escapeHtml(doc  
<td>${escapeHtml(document.creado_en)}</td>  
<td><a class="file-link" href="${escapeHtml(d  
</tr>
```

```
require_once __DIR__ . '/conexion.php';  
$conexion = obtenerConexion();  
$consulta = $conexion->query(  
    'SELECT id, nombre, especialista, indicacion, url_ficticia, creado_en  
    FROM documentos_medicos'
```

6. Posibles riesgos o mejoras pendientes

Se podría mejorar el formato de la fecha para que sea más fácil de leer y adaptar correctamente la zona horaria.

Explicación

HTML construye la estructura de la interfaz que ve el usuario. JavaScript se encarga de solicitar los documentos a la API mediante `fetch()`. PHP recibe la solicitud y utiliza PDO para conectarse con MySQL. MySQL almacena los documentos médicos y devuelve los datos solicitados. PHP procesa esos datos y los devuelve al frontend en formato JSON. Finalmente, JavaScript recibe la respuesta y utiliza esos datos para actualizar la tabla que aparece en el HTML.